Shiv Nadar University, Chennai

Department of Information Technology

25IT1731 Computational Thinking and Problem Solving with Python

Ex 3 – Functions and Recursions

Part A

1	Write a function *convert_units(value, unit)* that converts a distance value from kilometers to both miles and meters, returning both converted values.
2	Write a function *calculate_bill(units, rate=8)* to calculate an electricity bill. The function should compute the energy cost as units × rate. If the units consumed exceed 300, add an additional 10% surcharge on the energy cost (10% of units × rate). Then add a fixed meter maintenance charge of ₹50 to get the final bill amount and return it. Call the function twice: using only the units (default rate of ₹8 applied) and using units and a different rate, with units above 300
3	Write a function *book_ticket(name, num_tickets, price_per_ticket=250)* that calculates the total cost of movie tickets booked, where the price per ticket has a default value of ₹250. The function should return the customer's name and the total amount payable. Call the function using keyword arguments, in an order different from the function definition, once using the default price and once with a different price specified.
4	Write a function *statistics(*numbers)* that accepts a variable number of numerical values and returns the total, average, maximum, and minimum, calculating each using a loop over the arguments directly (do not store intermediate values in a separate list).
5	Write a Python program with the following functions: *count(n)* – returns the number of digits in n, *is_armstrong(n)* – checks whether n is an Armstrong number. The function *is_armstrong()* should call *count()* to determine the number of digits, using only integer variables For example, 9474 is an Armstrong number because $9^4 + 4^4 + 7^4 + 4^4 = 9474$.
6	Write a recursive function *factorial(n)* to calculate the factorial of a number. Using this function, calculate the sum of the series: $$\frac{1}{1!}+\frac{2}{2!}+\frac{3}{3!}+\cdots+\frac{n}{n!}$$
7	Write a recursive function *fibonacci_recursive(n)* to find the nth term of the Fibonacci sequence.
8	Write a function *find_lcm_hcf(a, b)* that returns both the LCM and HCF of two numbers, using an iterative approach (a loop), where the LCM is calculated using the formula LCM(a, b) = (a × b) / HCF(a, b). Do not use recursion or Python's built-in math.gcd()/math.lcm() functions.

9	Write a recursive *function hcf_recursive(a, b)* to find the HCF of two numbers using the Euclidean algorithm. Using this function, write another function lcm_hcf_recursive(a, b) that returns both the LCM and HCF
10	Write a recursive function *decimal_to_binary(n)* to convert a positive decimal integer into its binary representation. Example: Input: 25, Output: 11001

Part B

11	Write the following functions for online shopping application: • *calculate_subtotal(*prices),* accepts a variable number of product prices and returns the subtotal. • *calculate_discount(amount, discount=5),* calculates discount using a default discount rate of 5%. • *calculate_tax(amount, tax_rate=18),* calculates the GST using a default tax rate of 18%. • *generate_bill(...),* returns the discount, tax, and final amount.
12	Write a recursive function sum_of_multiples(n, divisor) that returns the sum of all numbers from 1 to n that are divisible by divisor, computed recursively (no loops, no lists). Example: sum_of_multiples(20, 3) is 3 + 6 + 9 + 12 + 15 + 18 = 63
13	A customer feedback app collects star ratings (1 to 5) from multiple customers for a product. The product manager wants to quickly find out how many customers gave a specific rating. Write a function *count_occurrences(target_rating, *ratings)* that returns the number of times target_rating appears among the given ratings, using a loop over the arguments (no lists, tuples, or dictionaries).

****